School of Computer Science and Engineering, UNSW

Never Stand Still

WK01-03: Blockchain Basics

Software Architecture for Blockchain Applications, COMP6452

Salil Kanhere, Nadeem Ahmed

Let's start with the basics of blockchains, going through the foundations of the technology.

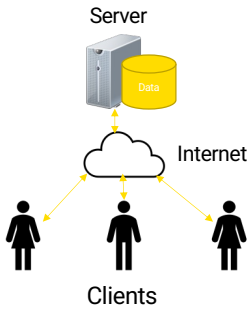
Agenda

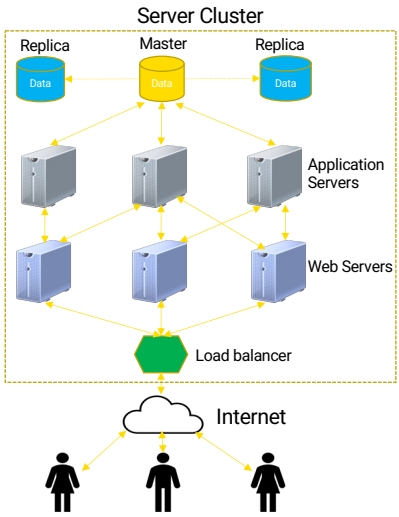
- Introduction to Distributed Ledger Technology
- Decentralisation and Consensus
- Transactions, Blocks and Ledger Structure
- Designing Blockchain Based Systems

Centralised Technologies

Centralised systems
- Client-server

Physically/logically
- Computation
- Data
- Administration

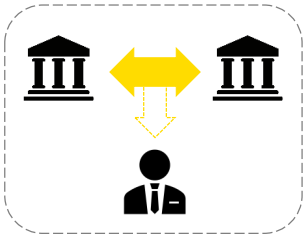




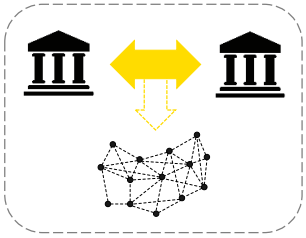
- Conventical technologies are essentially centralised systems.
- Among them most popular is the client-server model, which is an application structure that partitions tasks or workloads between the providers of a resource/service, called servers, and service requesters, called clients.
- Clients connect to the server through a network like the Internet.
- Clients may also perform some tasks like validating data, but the core of the application runs on the server.
- This structure can be found in many applications such as course management systems like WEBCMS or Moodle used by our university.
- The server may also use a centralized database to store data.
- Thus, both data and computation are centralized.

- This simple setup may expand into a massive server cluster consisting of multiple (1) web servers, (2) application logic servers, and (3) database servers running in one or more data centres.
- Where there are multiple copies of databases, usually data are written only to a single instance called the Master and then replicated into other databases for fast read access and high availability.
- While this setup is a lot more distributed than a simple client-server setup, the overall system still has logically centralized data and computation capabilities.
- It is also administratively centralised under a single organisation, e.g., Amazon and Facebook.
- Thus, conventual technologies mostly centralise data, computation, and administration.

Blockchain Technology



Centralised Trust using a 3rd Party



Decentralised Trust using a Blockchain

A distributed ledger that utilises the link list data structure

Main functionally:

- Shared append-only database (a ledger)
- Shared compute platform ("smart contracts")

Logically centralises data; administratively decentralises control

Blockchains are good for …

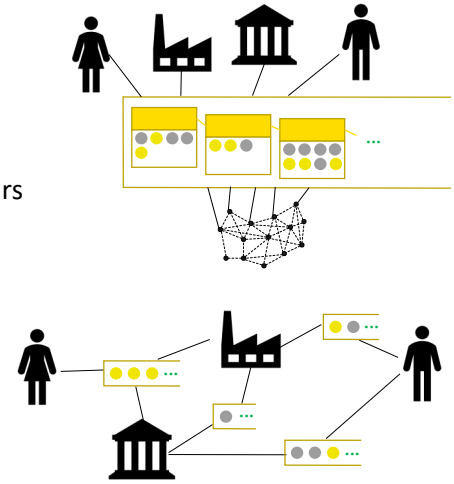
- New trustworthy & efficient ways to work together
- Exclusive control of digital assets

- Traditionally, when 2 organisations are in business together, often they need to agree to rely on some trusted 3rd party to facilitate the operation of their business relationship.
- In a blockchain, instead of having to trust some 3rd party, we can choose to trust the decentralised technical infrastructure that provides a shared ledger.

- To do so, blockchains offer 2 main functions:
  - They are a shared database or ledger to record TXs.
  - They are a shared computation platform for executing smart contracts (SCs).
- A blockchain is a distributed Ledger that is structured into a link list of blocks
- We are already very familiar with databases like relational databases and compute platforms like the cloud.
- But blockchain is different because it is a distributed system and doesn't have a central owner or operator.
- Even though it is distributed, the technical infrastructure ensures that there is consensus among all parties about the ledger and the computation. So we can say that blockchains logically centralise data but administratively decentralises control.

Blockchain vs Distributed Ledger Technology (DLT)

- Blockchains are one kind of distributed ledger
  - Ledger is an append-only list of blocks of transactions (TXs)
    - Ledger structure is one list, but operators are in a peer-to-peer network
- Distributed ledger systems share ledgers
  - Can be lots of small ledgers
    - Perhaps only parties of interest see TXs
  - Many possible kinds of ledger structures



- When the ledger that stores data is distributed, we call it a distributed ledger. However, compared to multiple databases in a client-server system where data is written only to the master (ready from master and replicas), these ledger copies can be read/written at any node.
- The Set of technologies around this design is called distributed ledger technology (DLT).
- So blockchain is one such realisation of a distributed ledger.

- A distributed ledger system/ecosystem shares multiple ledgers. E.g., 2 parties engaged in a supply chain may run a pairwise ledger and the supply chain may consist of multiple such ledgers.
  - In the bottom-right figure, small horizontal rectangles illustrate (small) ledgers shared between a few parties of interest.
- As we see in the next session, blockchains can be built in various ways while retaining the idea of a chain/list of blocks. Therefore, a distributed ledger ecosystem may even include multiple kinds of ledgers.
- All blockchains are DLT but not all DLTs are Blockchains

Distributed Ledger Technology (DLT) - Types

	Permissionless (no authorisation required to perform a particular activity)	**Permissioned** (authorisation required to perform a particular activity(ies))
Public (accessible to the public for use)	Open competition by operators Fees for use Greatest transparency & security Can be slow	Preauthorised operators Fees for use
Private (accessible only to a limited group of users for use)	Preauthorised operators	Preauthorised operators Easiest to regulate Greatest privacy

See: ISO 22739:2020(en), Blockchain and distributed ledger technologies — Vocabulary

In this class, we adopt the ISO 22739:2020 standard terminology from https://www.iso.org/obp/ui/#iso:std:iso:22739:ed-1:v1:en
DLT systems can be broadly classified as:
1. Public and private based on technical excludability of users.

A public DLT is accessible to anyone.
Anyone can use a public BC network. Bitcoin and Ethereum are the best examples.
Public BC networks provide incentives for people to join, contribute computing power, and charge fees for the use.
A private DLT is accessible only to a limited group of users.
Usually, they also don't use an explicit incentive mechanism like paying TX fees.
They are more suitable for regulated industries, e.g., in the finance industry, it is essential to Know Your-Customers (KYC).
Hence, an approval process is used before giving access to a BC.
2. Permissioned and permissionless based on the ability to perform activities such as validating transactions and updating the protocol.

In a permissioned DLT, authorisation is required to perform a particular activity, such as issuing a TX and building a block.
Other forms of permissions include:

- Permission to initiate TXs, permission to mine.
- Also fine-grained permissions such as permission to create a particular asset and execute a function in a SC.
- Public BC networks can also be used for private purposes by setting access rights through smart contracts.
- In a permissionless system no such authorisation is required to perform any activity

Public-permissionless DLTs are the most well known, e.g., Bitcoin and Ethereum.

- They encourage open competition and involve TX fees.
- They provide the greatest level of transparency and immutability.
- However, they have poor performance in terms of TX inclusion and computing resource consumption.

Public-permissioned DLTs use preauthorised operators as miners/validators.

- A bunch of BCs such as Hedera, Ripple, Avalanche, and Algorand exist in this space.
- They do charge fees for use.

Private-permissionless DLTs also has preauthorised operators.

- The best example is running a public BC like Ethereum within a private network.

Private-permissioned are another set of popular DLTs that has preauthorised operators.

- A bunch of DLTs such as Hyperledger Fabric, R3 Corda, and VMware Concord exists in this space.
- Because they are private and permissioned they are the easier to regulate and they provide the greatest privacy.
- They do tend to provide better performance too.

You may also come across the term “consortium DLTs/BCs”. They are essentially private DLTs used in cross-organisational contexts. There is usually no difference in the technology used in such cases.

Distributed Ledger Technology (DLT) - Types

	Permissionless (no authorisation required to perform a particular activity)	**Permissioned** (authorisation required to perform a particular activity(ies))
Public (accessible to the public for use)	Bitcoin Ethereum Algorand	Hedera Ripple Avalanche
Private (accessible only to a limited group of users for use)	Ethereum on a VPN	Hyperledger Fabric R3 Corda Quorum Hyperledger Besu VMware Concord

- BC platform/framework software is usually open source. Thus, we also see many different instances of the same BC framework. For e.g., many public and private BC instances are based on Ethereum and EOS
  - A bit of terminology to be consistent, but following 2 terms are used interchangeably:
    - BC platform – An instance of a blockchain, e.g., Ethereum and Bitcoin. While Ethereum mainnet and Ethereum Goerli testnet are similar in functionality, they are 2 separate networks/instances.
    - BC framework – Software code used to launch a blockchain instance, e.g., Hyperledger Besu & Quorum are 2 codebases for launching a private-permissionless Ethereum-like network.
    - BC network – Interconnected set of nodes/miners that maintain the ledger and validate the TXs.
- Also, we see many examples of partial code reuse, e.g., Ethereum-style SCs are supported in Binance Smart Chain (BNB), Hyperledger, and VMware Concord.
- When public BC platforms are used as private or consortium BCs, TXs fees could be ignored and access control could be provided via a firewall.
- We even see applications that use a mix of all 4 types of BCs.

Blockchain System

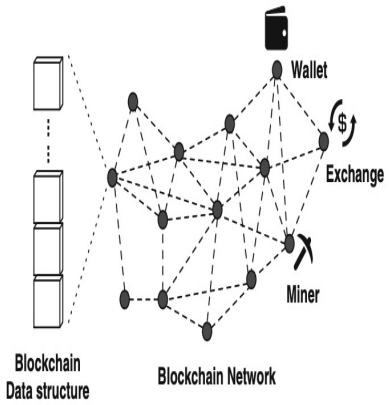
1. A blockchain network of nodes

2. A blockchain data structure for the ledger that is replicated across the network (full nodes)

3. A network protocol that defines rights, responsibilities and means of communication, verification, validation and consensus
   - Includes authorisation and authentication
   - Incentive mechanism
   - Mining

4. Wallets

5. Exchanges

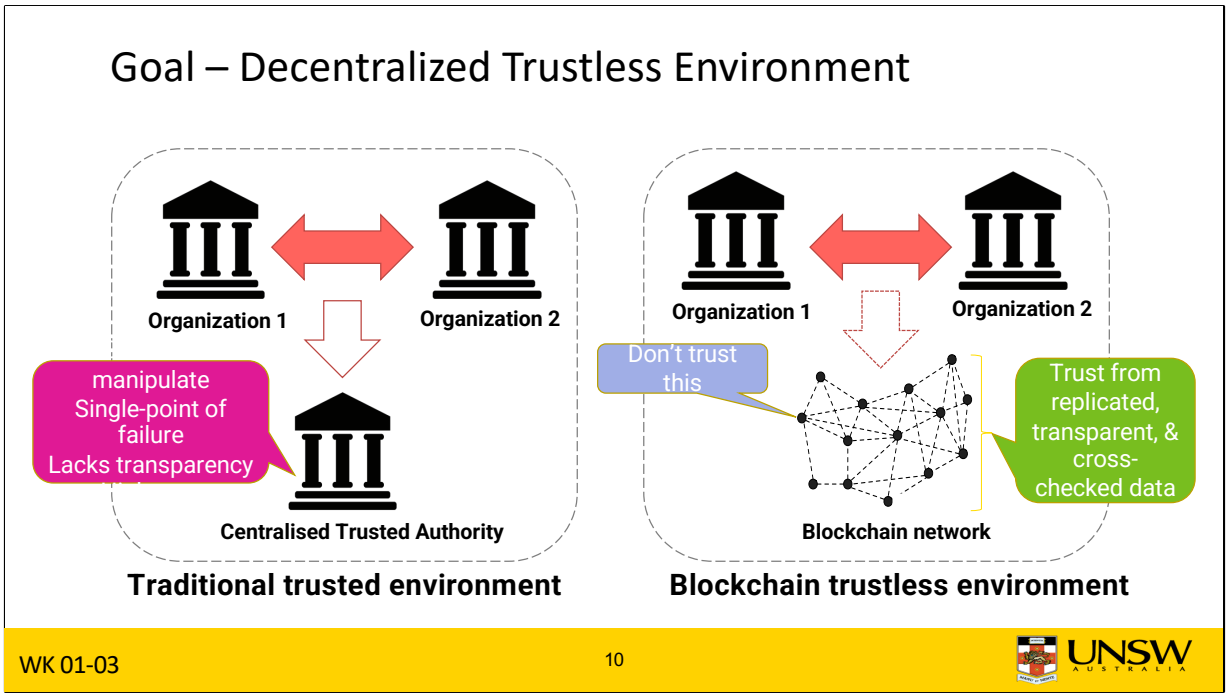


A Wallet maintains the cryptographic key pairs – public and Private keys
Exchange facilitate the trading of Crypto currencies in exchange for other cryptocurrencies or fiat currencies

Agenda

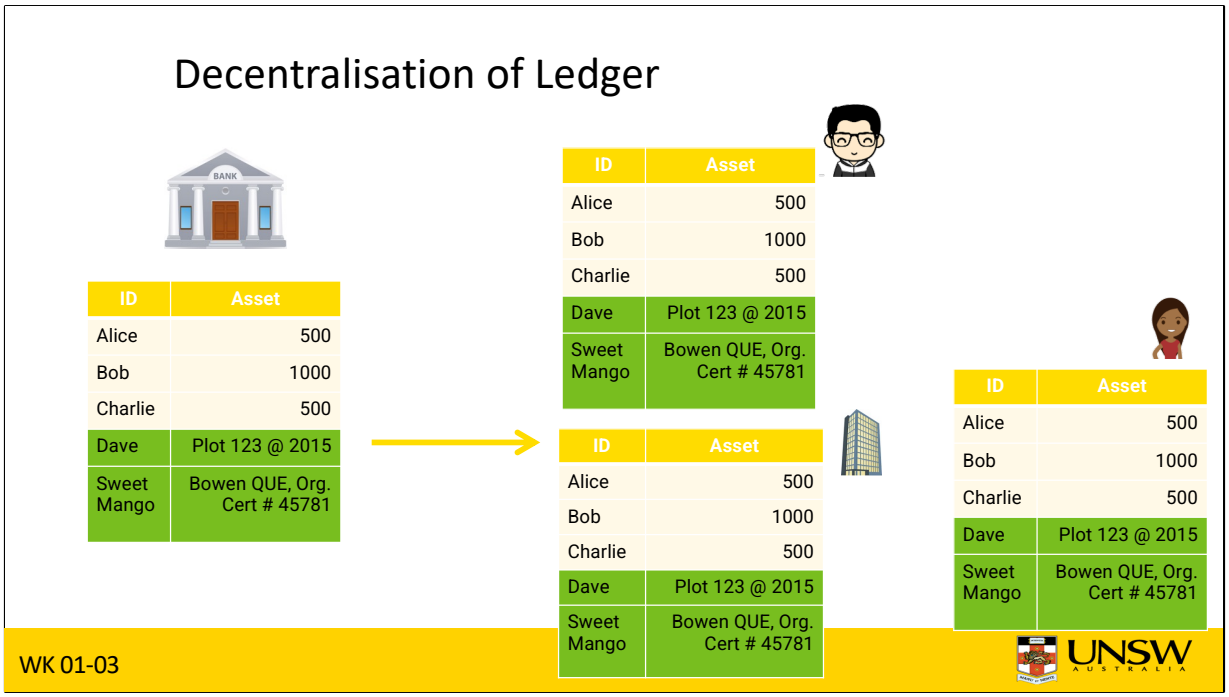
- Introduction to Distributed Ledger Technology
- Decentralisation and Consensus
- Transactions, Blocks and Ledger Structure
- Designing Blockchain Based Systems

Let's discuss what these terms mean using a few examples. These help us to understand the problem that BCs/DLTs really solve

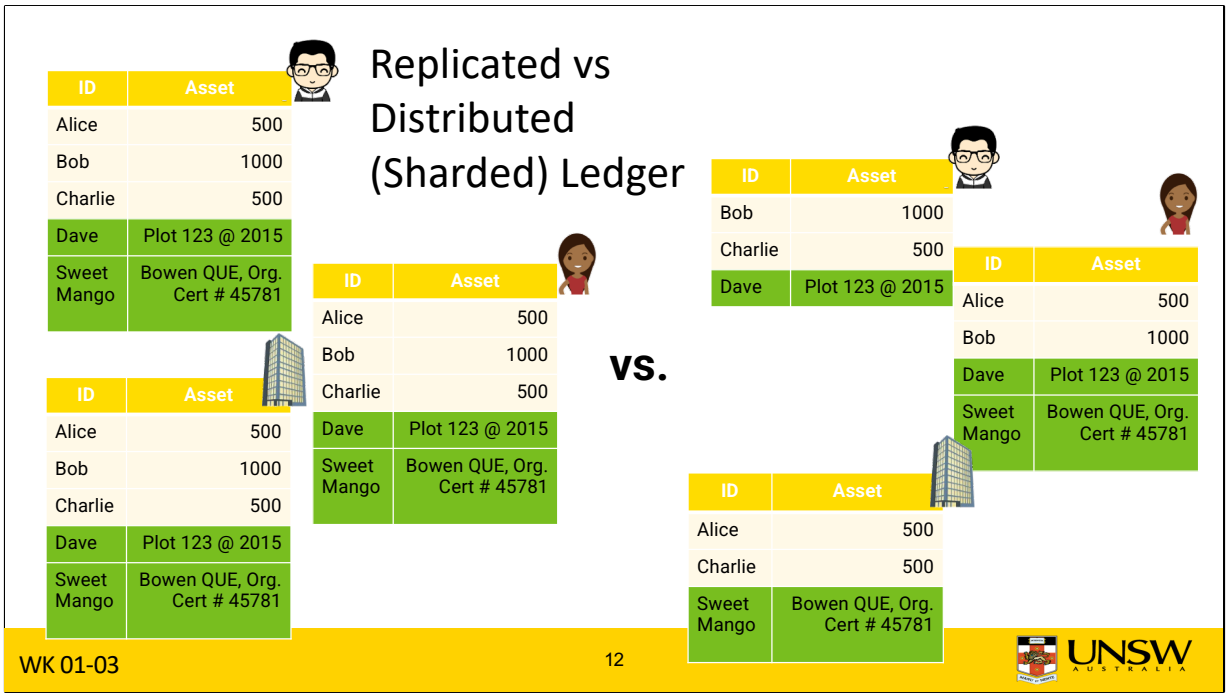


- BC's goal is to replace the central authority with a decentralised solution based on a set of computers.
- While central trusted parties such as banks, regulatory bodies, and government are essential to establish trust between 2 transacting parties, they are many concerns around them. For example,
  - They have the power to control and manipulate the system.
  - Lead to a single point of failure.
  - Their internal system state is opaque to the participants.
  - They are fragmented making it difficult to interoperate and collaborate.
  - Leading to high costs.
- When it comes to the BC network, we don't trust any single computer but trust their collective behaviour.
- We ensure correct collective behaviour by:
  - Replicating the state across many nodes in the network.
  - Ensure all nodes agree on what changes are made to the state, and by whom. Cross-checking each other's computations.
  - Making the state public.
  - Also, incentives may be given to nodes to ensure the majority behave correctly.

- While this appears easy, let's discuss some requirements in replacing the authority and challenges we need to overcome in doing so.

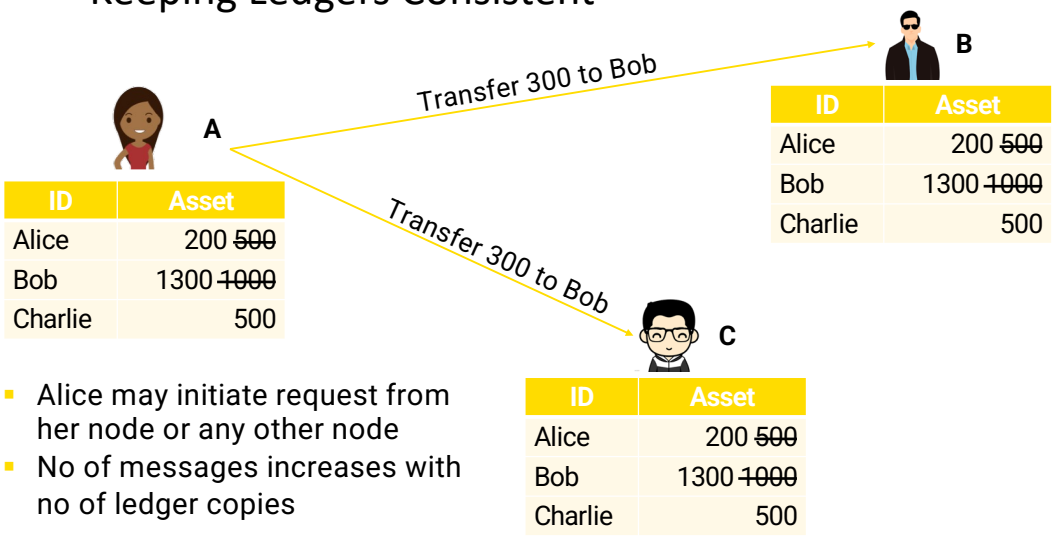


- We are looking for a decentralised design by distributing the ledger maintained by the central authority.
  - The ledger maintains asset ownership of Alice, Bob, and Charlie. The amounts here could be fiat or cryptocurrency.
  - Note that a ledger can keep track of other forms of data like ownership of land or goods in a supply chain.
- This sort of design breaks its monopoly, enhances transparency, and avoids single-point of failures.
- Each copy of the ledger is now managed by a different party.



- We also have a choice of whether we want to have a fully replicated or distributed ledger.
- On the left, we have a fully replicated ledger where 3 parties maintain the same data while on the right, we keep only 2 copies of the same piece of data
- While the design on the left leads to a very robust solution against failure, it's costly to maintain and reduces the write performance. For e.g., today, Bitcoin ledger is ~ 660 GB while Ethereum ledger is 1315 GB
    - See https://ycharts.com/indicators/bitcoin_blockchain_size and https://ycharts.com/indicators/ethereum_chain_full_sync_data_size
- While most contemporary BCs adopt full replication, BC such as R3 Corda and BigchainDB adopt a decentralised design.

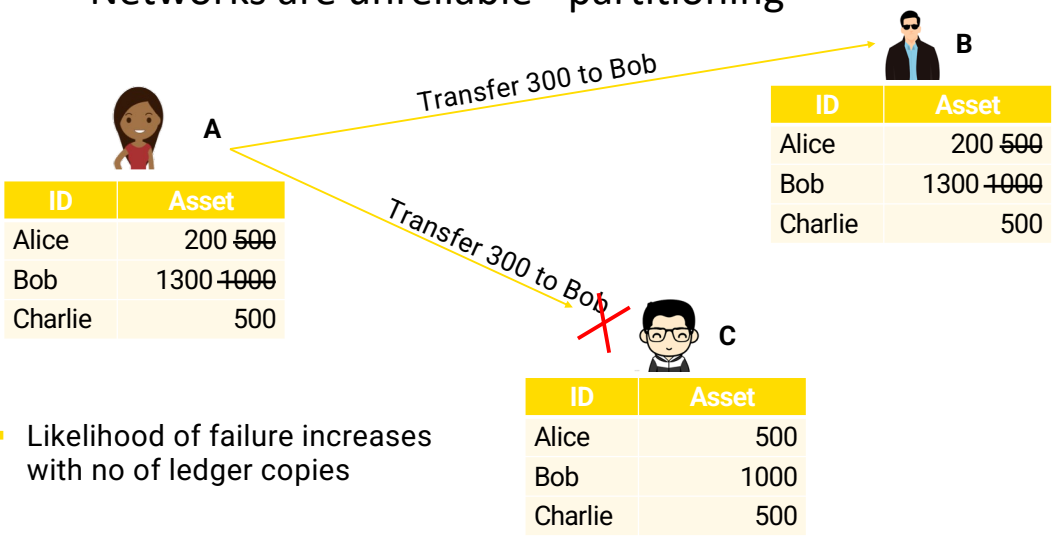
Keeping Ledgers Consistent



- Alice may initiate request from her node or any other node
- No of messages increases with no of ledger copies

- Whether it's a replicated or distributed ledger, as soon as you have more than one copy, we need to worry about keeping the ledger consistent.
- Suppose we keep 3 copies of the ledger and Alice wants to transfer $300 to Bob.
- Ideally, this should be recorded on all 3 copies where $300 deducted from Alice and added to Bob's account.
- This can be achieved using message passing, and the number of messages needed increases with the number of ledger copies.

Networks are unreliable - partitioning



ID	Asset
Alice	200 ~~500~~
Bob	1300 ~~1000~~
Charlie	500

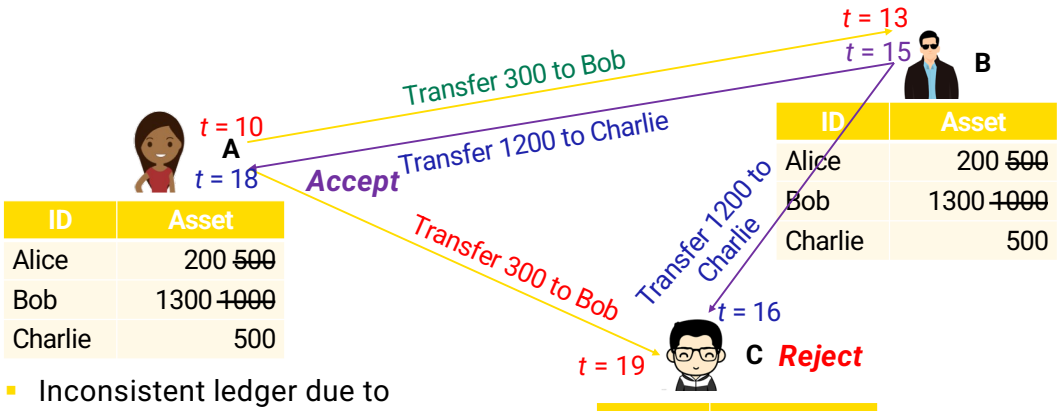
ID	Asset
Alice	200 ~~500~~
Bob	1300 ~~1000~~
Charlie	500

ID	Asset
Alice	500
Bob	1000
Charlie	500

- Likelihood of failure increases with no of ledger copies

- Unfortunately, networks are unreliable (referred to as network partitioning) and messages may get dropped. Hence, while Alice's message may reach node B, it may not reach node C leading to an inconsistent ledger.
- This problem worsens when there are many copies of ledger as more messages are needed and a few of them may get dropped.
- While a temporary inconsistency may be ok, as far as we can reconcile later once the message is received, this could lead to other problems.

Concurrent Transactions



ID	Asset
Alice	200 ~~500~~
Bob	1300 ~~1000~~
Charlie	500

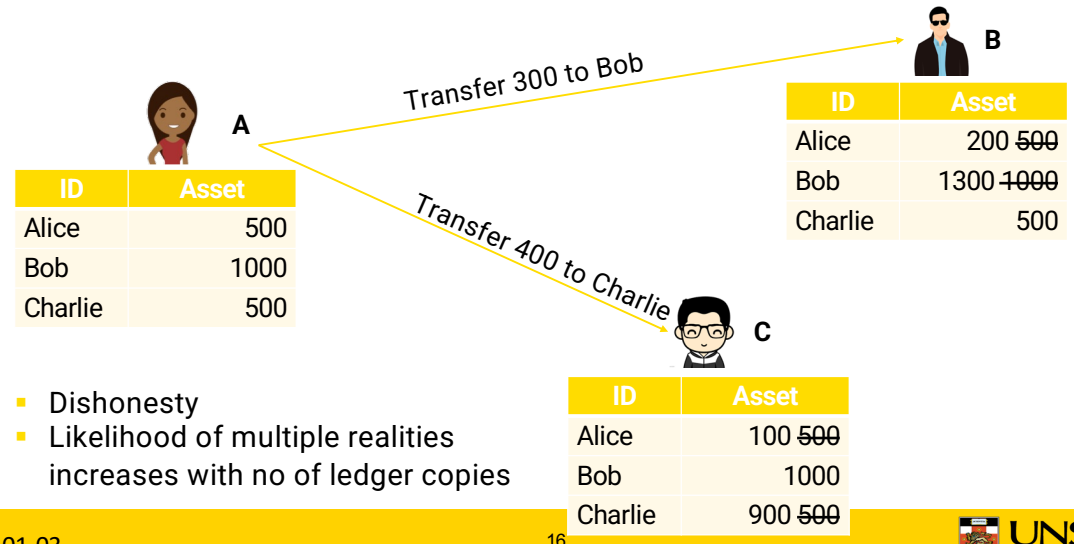
ID	Asset
Alice	200 ~~500~~
Bob	1300 ~~1000~~
Charlie	500

ID	Asset
Alice	500
Bob	1000
Charlie	500

- Inconsistent ledger due to concurrent TXs
- Potential for inconsistency increases with no of ledger copies

- Another case that can go wrong is when TXs are concurrent.
- Based on Alice's TXs to transfer $300 to Bob, Bob issues another TX to transfer $1,200 to Charlie.
- However, due to varying network delays across nodes (particularly on the Internet) or transient failures in nodes, C may receive Bob's TX before Alice's TX.
- In that case, it will reject Bob's TXs leading to an inconsistent ledger.

Double Spending Problem

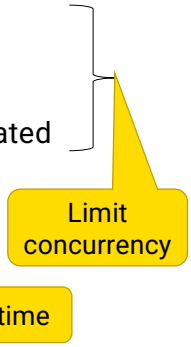


- Dishonesty
- Likelihood of multiple realities increases with no of ledger copies

- If the ledger synchronisation is designed to accommodate inconsistencies due to network limitations, Alice may benefit from temporary inconsistency to double-spend her money.
- E.g., while Alice transfers $300 to Bob, she also transfers $400 to Charlie to buy a bicycle.
- Even though Alice doesn't have enough money for both TXs, ledger copies will accept each TX, as per their copy of the ledger Alice has enough money.
- While the inconsistency will be eventually detected and one or both TXs could be reverted, Charlie may have also shipped the bicycle Alice ordered.
- Like the network problem, the potential for multiple realities like double spending attacks increases with no of ledger copies.

Solutions to Overcome Consistency Issues

- What are some potential solutions to overcome consistency issues?
- Maybe?…
  - Keep only a single copy of an account/record
  - Get a confirmation/Ack from all the replicas before updating an account
  - Complete one TX on all nodes before allowing another TX on a related account
  - Wait some time for a TX to complete
  - Nodes to periodically double-check each other's state
  - Order TXs based on time
  - ….



- Can you think about several potential solutions to overcome the consistency issues we came across so far?
- While we can think of many solutions, most of them are related to ideas like the above.
- Most likely these ideas are centred around either preventing concurrency or relay on some notion of time such as waiting for some time or relying on the sender's time.
  - They limit decentralisation and rely on some form of sequential TXs while substantially slowing down TX processing.

Timing & Ordering

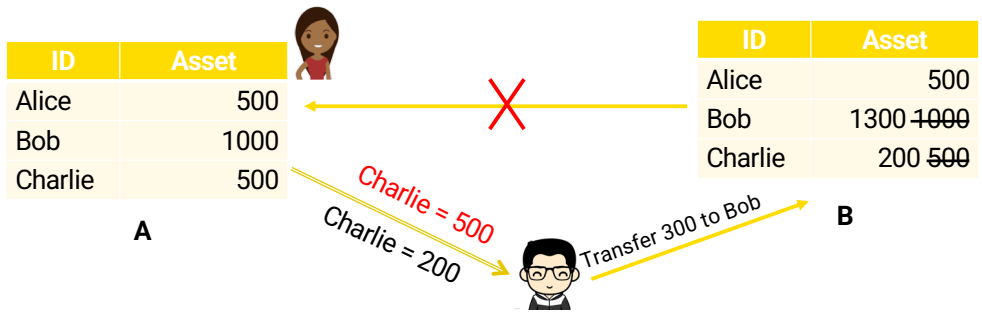
- Use of sender's time to order TX isn't reliable
  - Clocks aren't accurate – Drift & skew
  - Difficult to sync them & keep them synched
  - Sender could change time to game the system
- Physical time isn't a reliable measure in distributed systems
- We need a way for nodes/replicas to agree on a global ordering of TXs

- Unfortunately, time on a distributed system is not a reliable measure.
- Some clocks are fast, some are slow, and even if you sync them, they lose the synchronisation after a while.
  - The difference in the rate that clocks tics is called the clock *Drift.*
  - The difference in time of a set of clocks is called the clock *Skew.*
- Also, if we use time, Alice could also manipulate the timestamp she attaches to each TX leading to further chaos.
- Essentially, physical time isn't a reliable measure in distributed systems. Due to this issue, most distributed systems protocols are designed without any assumption in time, and blockchain is no different.
- However, if we look at what we need is for the nodes/replicas to agree on a global ordering of TXs. In the Concurrent Transactions slide, rather than worrying about whether the TX transferring $300 or $1200 is the 1st one, what matter is both nodes B and C agree on one of them being first.

CAP Theorem

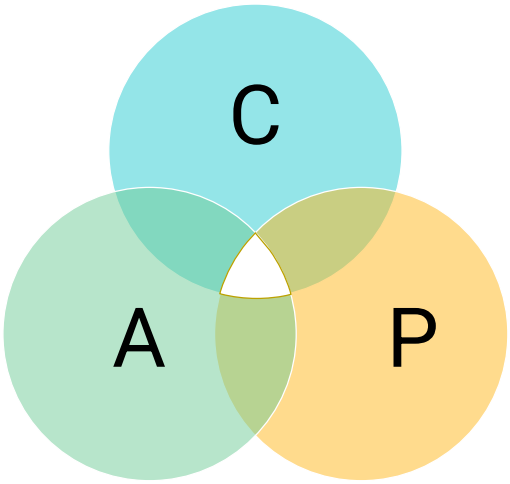
"It's impossible for a web service to provide following 3 guarantees at the same time" (Eric Brewer, 2000)

- Consistency
- Availability
- Partition-tolerance



- Another concept related to unreliable communication is the CAP theorem.
- In 2000, Dr. Brewer said that "It's impossible for a web-based service to provide guarantees on consistency, availability, and partition tolerance at the same time"
  - Formally proven in Seth Gilbert and Nancy Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services", *ACM SIGACT News*, Volume 33 Issue 2 (2002), pg. 51–59. doi:10.1145/564585.564601.
  - This eventually became a theorem and was formally proved a year later.
- Let me illustrate the idea using an example.
- Charlie transfers $300 to Bob and sends the message to replica B, which in turn is expected to update replica A.
- So if Charlie queries A, he should get a balance of $200.
- However, if the network is partitioned such that replicas A & B can't talk to each other, when Charli queries replica A it'll get an inconsistent result.
- So we can see that while the distributed ledger is available for reading and writing and the network is partitioned, the ledger state is no longer consistent.
- The only way to keep the ledger consistent during network partition is to stop updating the ledger. Which means the availability is affected.

CAP Theorem - Cont.



Choose any 2

But networks will partition!

- Nodes loose network connectivity
- Partition tolerance is mandatory in distributed systems

So, choose Consistency or Availability…

- Blockchain needs to be Available
- Ledger needs to be Consistent
  - Consensus → Consistency

- CAP theorem essentially says to choose any 2 out of 3.
- However, partition tolerance is not really a choice, as nodes loose network connectivity occasionally, especially in the Internet.
- Hence, partition tolerance is mandatory in distributed systems.
- So, choice is either to compromise consistency or availability.
- As discussed so far, BC needs to be available, and the ledger needs to be consistent. Consistency is achieved by consensus which is very difficult.
- We are asking for 2 conflicting and difficult properties. We will later see how most public blockchains retain availability while hoping consistency will be achieved eventually.

Consensus

Agreement in the judgment or opinion reached by a group as a whole
- Getting 2 nodes to agree is hard
- Getting many nodes to agree is even harder

Achieving consensus under:
- Unreliable communication – Partition tolerance
- Failed nodes – Fault tolerance
- Misbehaving nodes – Byzantine fault tolerance

Consensus can be achieved algorithmically with various trade-offs

- Agreement in the judgment or opinion reached by a group is referred to as *consensus.*
- In the real world, we know how hard it is for 2 people/nodes to agree. And this just gets worse as we add more nodes into the equation.
- Hence, achieving consensus on the order of TXs and state in a distributed ledger is extremely difficult in the presence of:
    - Unreliable communication. The ability to tolerate such issues is called *Partition tolerance.*
    - Fault tolerance is needed to handle Failed nodes.
    - Misbehaving nodes need more complicated solutions such as Byzantine fault tolerance.

- Ultimately, different blockchain designs present ways to tolerate these failures/issues and algorithmically achieve consensus. Of course, they come with various trade-offs.

Agenda

- Introduction to Distributed Ledger Technology
- Decentralisation and Consensus
- Transactions, Blocks and Ledger Structure
- Designing Blockchain Based Systems

Transactions (TX)

Identifiable data package
- Monetary value (Cryptocurrency)
- Code (Smart Contract)
- Parameters/results of function calls

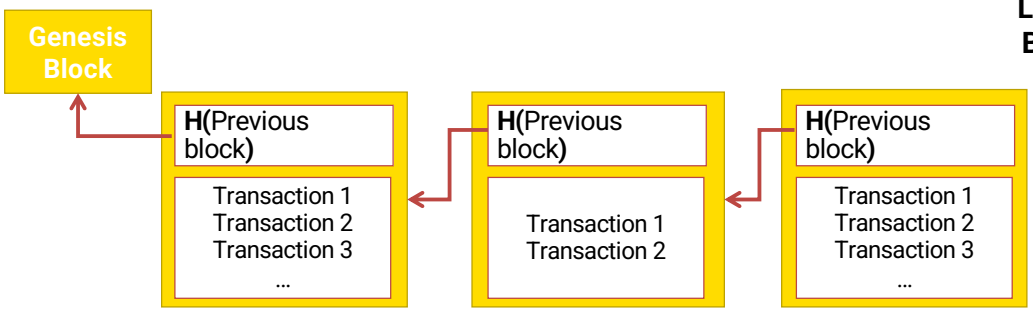
To:
From:
Value/Data:
Sender's Signature:

```
Input:
Previous tx: f5d8ee39a430901c91a5917b9f2dc19d6d1a0e9cea205b009ca73dd04470b9a6
Index: 0
scriptSig: 304502206e21798a42fae0e854281abd38bacd1aeed3ee3738d9e1446618c4571d10
90db022100e2ac980643b0b82c0e88ffdfec6b64e3e6ba35e7ba5fdd7d5d6cc8d25c6b241501

Output:
Value: 5000000000
scriptPubKey: OP_DUP OP_HASH160 404371705fa9bd789a2fcd52d2c580b65d35549d
OP_EQUALVERIFY OP_CHECKSIG
```


- A TX is an identifiable data package or a unit of operation.
- Depending on the BC platform, it may consist of a monetary value, SC code, or parameters/results of SC function calls.
- As with a TX in physical world (e.g., bank transfer), there is a sender, receiver, amount, and authorisation in the form of a signature.
- Such fields in a TX such as to, from, and signature are compulsory.
- This is an example from Bitcoin, which define a set of binary input and outputs.

Blockchain Formation



- Collection of ordered TXs form a block's body
- Summary of those TXs & hash of previous block forms a block's header
- Collection of blocks form a blockchain
  - Blocks are built by a distributed set of nodes, aka mining
  - Nodes agree on contents of a block, order of blocks, & who has the right to build a block → Consensus to achieve consistent chain of blocks

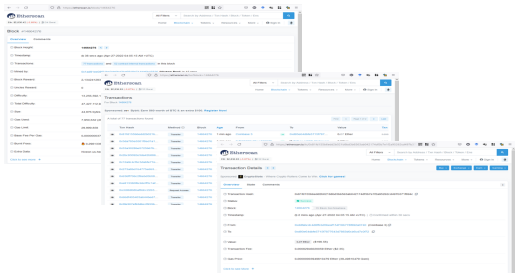
- A collection of ordered TXs form the body of a block.
- A block also refers to a unique fingerprint of the previous block called the *hash*.
- A summary of those TXs and the hash of the previous block form the header of a block.
- The very first block is called the *genesis block*.
- The collection of blocks forms a BC.
- The process of building a block is called mining.
- The nodes in the BC network agree on the contents of a block, the order of blocks, and who has the right to build a block – This is essentially the consensus process that leads to a consistent chain of blocks.

Let's take a step back …

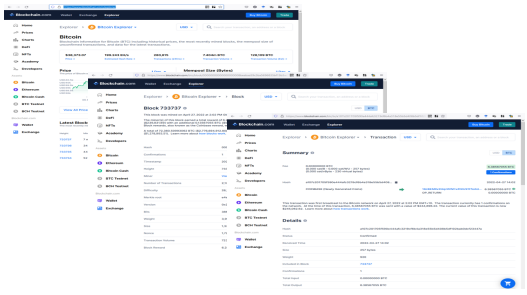
- Live Demo of TXs, Blocks, Blockchain
  - https://andersbrownworth.com/blockchain/

- Blockchain explorers

e.g., etherscan.io

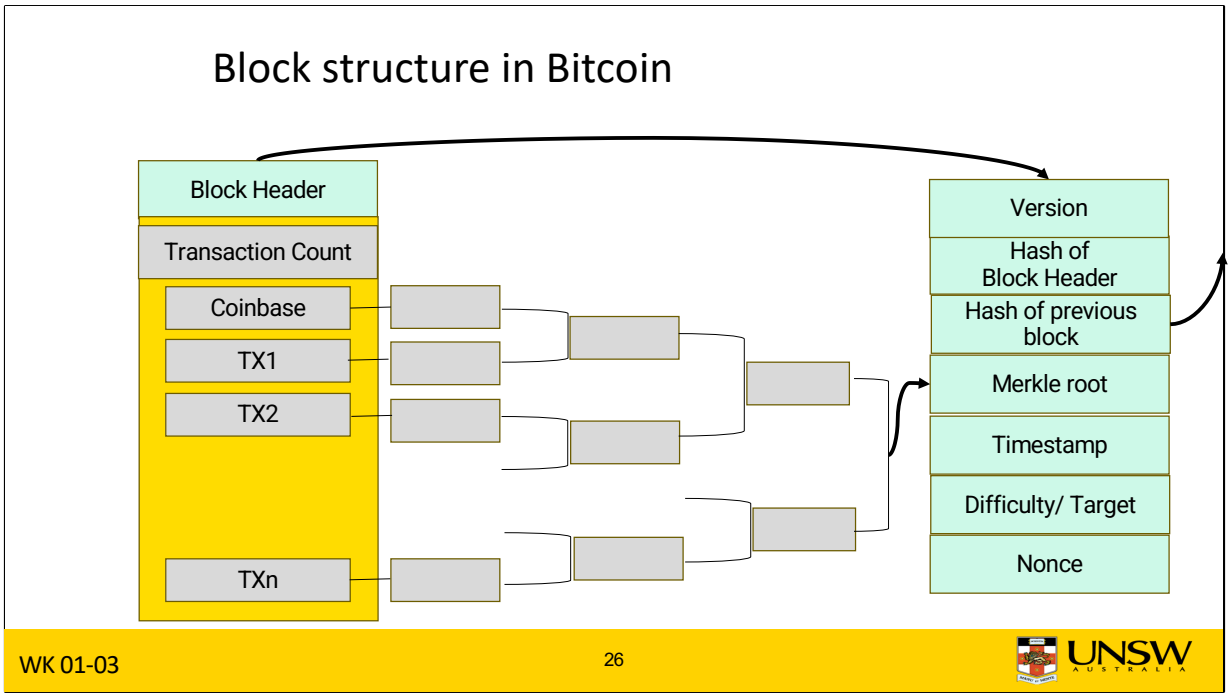


e.g., blockchain.com/explorer



Highly recommended to watch the entire videos by Andres Brownworth

- Blockchain explorers are like search engines for blockchain TXs.
- They can be used to explore various data about blocks, their TXs, and smart contracts about one or more blockchain platforms like Bitcoin, Ethereum, Litecoin, and Ripple.
- There are many explorers and some of the popular ones are http://eherscan.io, http://blockchain.com/explorer, and http://blockchair.com



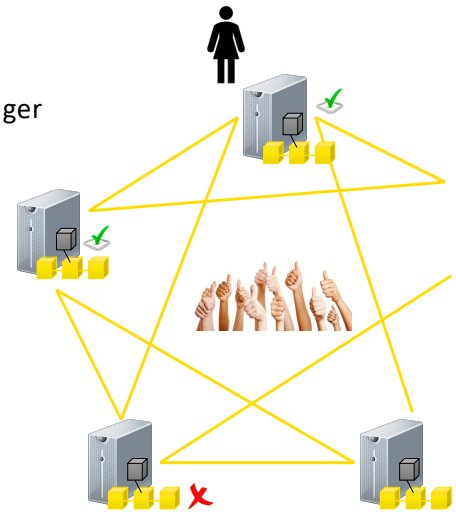
This is the structure of a block in Bitcoin
We will go through the details in future lectures.
You can easily relate to a few parameters here.
Nonce should be above the Target value that specifies the difficulty of finding acceptable PoW.
Coinbase is the first Transaction that contains the block rewards (mining) + the transaction fees
Note the Merkle Tree elements (leave nodes, intermediate nodes and Merkle root).

Network Infrastructure

Peer-to-Peer network of nodes
- Each usually has same rights to update the ledger

Each node hosts a replica of the ledger
- High availability
- Efficient read access

Communication
- Gossip protocol for TX propagation
- Consensus protocol for agreement



- Let's talk a bit about the BC network.
- It's essentially a peer-to-peer (P2P) network where all participants have the same rights to store and manipulate data.
- Because there are so many nodes and copies of the ledger, we can achieve high availability and fast reading, as you can access data from any of them.
- TXs are propagated among P2P nodes based on a gossip protocol.
    - Algorithmic gossiping is not very different to gossiping in real-world where a friend tells you some gossip, and you tell it to someone else. Eventually, it's no longer a gossip and everyone knows it except a few colleagues…
- Further, nodes use a consensus protocol to agree on what changes are to be made and by whom.
    - Typically, super-majority agreement (2/3) is used to determine what changes are to be made.

Question

Which of the following statement(s) is True about blockchains?

A. Each node in a blockchain is trustable

B. The process of building a block is called mining ✓

C. Permissions can't be used with public blockchains

D. A transaction can be altered by modifying only the block that contains the transaction

- Answer is B
- Each node in a blockchain is trustable – We don’t trust individual nodes, but their collective behaviour
- The process of building a block is called mining – Well we just touch this topic, but others being not correct this is the only answer
- Permissions can't be used with public blockchains – We can have public-permissioned BCs
- A transaction can be altered by modifying only the block that contains the transaction – Nope, you need to change all the subsequent blocks

Agenda

- Introduction to Distributed Ledger Technology
- Decentralisation and Consensus
- Transactions, Blocks and Ledger Structure
- Designing Blockchain Based Systems

Let me give a very high-level overview of designing BC-based applications. We will discuss this topic in much details in future lectures and projects.

Designing Blockchain-based Systems

Many potential applications; each with specific requirements
- e.g., reveal quantity/volume bought, not the price
- e.g. announcements should be public vs. transactions should be "private"

Many challenges in designing systems for these various applications
- When to use a blockchain?
- What blockchain platform to use?
- How to resolve trade-offs?
  - e.g. Transparency vs. Privacy; Immutability vs. Propriety; Flexibility vs. Simplicity
- What data & function to handle on-chain, what off-chain?
- How to integrate with what other databases, networks, services, …?

- While there are many exciting applications of BC, especially in lack-of-trust settings, BCs and applications that could benefit from them are quite complex.
- There are many potential application of BCs beyond cryptocurrency and digital finance, e.g., supply chain, real estate, data sharing, digital rights / intellectual property management, and voting.
- However, these applications comes with specific, and sometimes conflicting requirements, e.g.,
  - In a supply chain we want to reveal qualities but not the price of good.
  - Exitance of an asset may be public but not its current owner or how it changed hands.

- As we'll explore more details around BCs, you will realise that it's nontrivial to answer some of the important questions like:
  - When to use a BC?
    - If you can solve a problem better with conventional technologies, don't use a BC. This could be considered 1st law of BC ☺
  - What BC platform to use?
  - How to handle trade-offs in your solutions such as
    - Cost, latency, and confidentiality. Higher level of transparency enabled by BC is not in line with business confidentiality needs.
    - Immutability is a problem when we want to fix errors.
      - Propriety – state of being correct.
    - BCs are somewhat rigid and not the most trivial to understand or work with.
    - Lack of quality of service guarantees & difficulty in correcting errors.
  - What to handle on the BC, what to keep off the BC?
  - How to integrate with other BC networks and conventional information systems?

- Therefore, in this course, we take a software architecture-related approach to designing BC-based applications to achieve the desired functional and non-functional requirements.
- That'll help us answer these questions more objectively without being biased by so many unfounded claims around BC technologies.
- You will also see that we pay greater emphasis on non-functional aspects, as they are important in architectural design to address trade-offs such as performance, quality of service, and cost

Properties of Blockchain

Non-functional properties of blockchains compared to conventional centralised technologies:

- (+) Integrity, Non-repudiation
- (+ read/ - write) Availability
- (+ read / - write) Latency
- (-) Confidentiality, Privacy
- (-) Modifiability
- (-) Throughput / Scalability / Big Data

Can combine blockchain with other system elements to compensate

- BCs can be differentiated from other conventional centralised systems based on a set of non-functional properties (we'll define these properties more formally in a later class):
  - Integrity – Prevention of unauthorised modification
  - Non-repudiation – Inability to deny some action
  - Availability – Be able to access when you need
  - Latency – Time is taken to perform an action
  - Confidentiality – Prevention of unauthorised disclosure
  - Privacy – A more general idea of sharing on a need-to-know basis
  - Modifiability – Ability to modify
  - Throughput - No of tasks/TXs performed per unit time
  - Scalability – The ability of a system to retain its performance with the increasing workload
  - Big Data – Large volume of data. Also be high velocity, variety, and veracity, aka 4Vs of big data
- Some of these limitations can be overcome by combining BCs with other systems and components.
- Also, throughput and scalability are improving with more advanced BC designs.

Smart Contracts (SC)

Some Blockchains can deploy & execute programs called "Smart Contracts" (SCs)

User-defined code, deployed on & executed by whole network

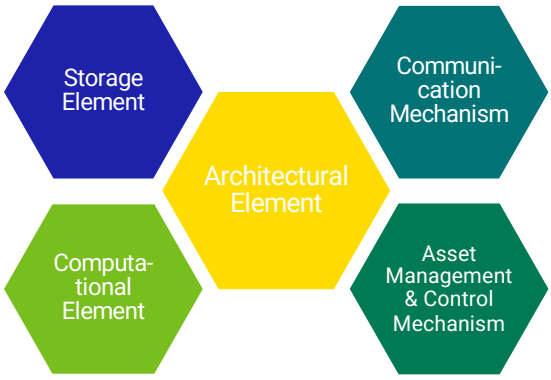
- Can be seen as a bunch of if/then conditions that are an algorithmic implementation of a business logic, e.g.,
  - Trading – Can hold & transfer assets/states, managed by the contract itself
  - Insurance – Can enact decisions on complex business conditions
  - SCs put counterparty obligations & rights in code, & blockchain nodes enforces those rights

Code is deterministic & immutable once deployed

- Its execution is triggered via a TX

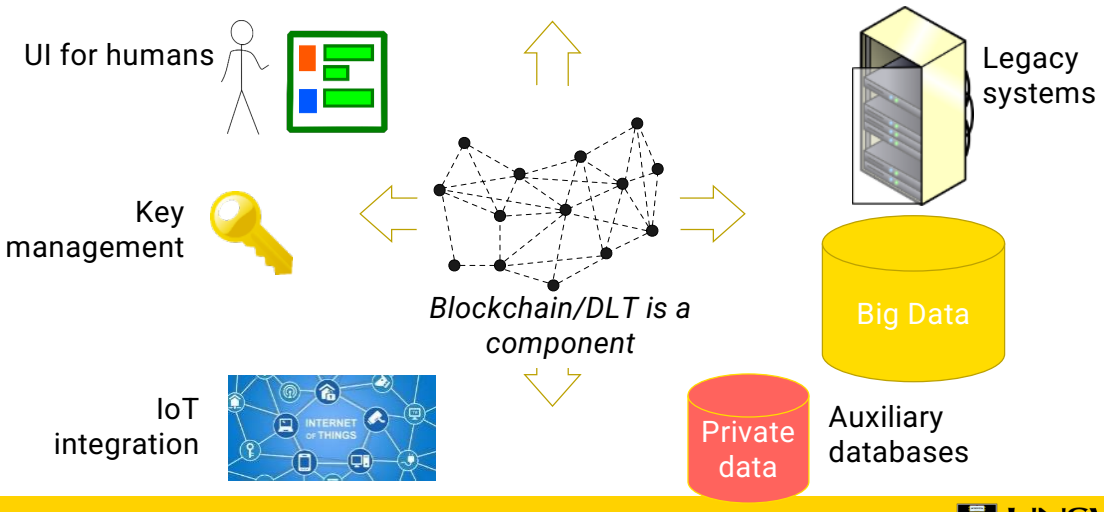
- While BC was the foundation for cryptocurrencies like Bitcoin, it has found much wider applications with the introduction of Smart Contracts (SCs).
- SCs are essentially small user-defined code/programs/scripts deployed & execute on BC nodes.
- SCs can be seen as a bunch of if/then conditions that are an algorithmic implementation of a business logic like financial service, e.g., trading, lending, and insurance.
  - In trading the SC code can hold & transfer assets/states, managed by the contract itself.
  - In insurance, the program logic can enact decisions on complex business conditions. E.g., automatic partial payment can be triggered as a product moves along a supply chain.
  - Smart contracts put counterparty obligations and rights of a business contract in code. The blockchain network of nodes enforces those rights.
- Due to the immutability of a BC, SC code is immutable once deployed. Given the same ledger state and external inputs, an SC always produce the same output making them deterministic too.
- Execution of an SC is triggered via a TX.
- There is a cost to deploy & execute. E.g., in Ethereum, we pay per assembler-level instruction executed by an SC.

Roles Blockchain can Play in Software Architecture



- There are 4 key functions of a BC as an architectural element in software architecture..
- Within an architecture of a software solution, a BC can be integrated as an element that provides storage and computation.
  - Storage elements – essentially as a database.
  - Computational element – As a computing engine that can run SCs.
- It could also be used as a communication mechanism for multi-party interactions – one party writes data into the BC that the other party can see, e.g., in supply chains.
- Its storage and computation aspects can be combined to provide asset management and control mechanisms too.

Are Blockchains Standalone Systems?



- BCs aren't stand-alone systems.
- They need to be integrated with many other systems to be practically useful, e.g., in supply chains they need to be connected with many information systems and sensors.
- Also, some sort of a UI is needed for user interaction.
- Cryptographic keys used to sign TXs need to be managed using a secure key management solution.
- BCs also need to integrate with external systems including legacy systems, e.g., inventory control and real-time status updates.
  - E.g., in a beef supply chain, we need to record the temperature experienced by beef during transport and storage using IoT integration.
- Also, it will be part of a large data ecosystem for both storage and data analytics. Private data can be kept off-chain..

What is the Reality?

TX behavior is difficult to predict

Immature Code!

TX failure difficult to detect

- Mistakenly retry

Unauthorised execution/termination of smart contracts

- 7% smart contract can be terminated without authority (Public Ethereum)*

Failure is permanent

- Decentralized Autonomous Organization (DAO) code issue led to USD 60 Million Ether theft during ICO (Initial Coin Offering)
  - Hard fork of Ethereum

*I. Weber, V. Gramoli, M. Staples et al., "On availability for blockchain-based systems", SRDS'17, Hong Kong, China, Sep. 2017.

- BCs have their caveats and the end result could be quite costly if proper design and controls are not in place.
- While SC code is deterministic, TX behaviour is difficult to predict as it depends on concurrent TXs.
- Also, being a distributed system failures are difficult to detect. E.g., a retry mechanism used to deal with network timeouts may end up generating duplicate TXs if it timeouts prematurely. As discussed earlier no one knows what a good time estimate on a distributed system is.
- Unauthorised execution and termination of SCs are another set of problems. E.g., 7% SC on Ethereum can be terminated without authority
- Immutability means failure is permanent and there is no way of going back.
  - E.g., the Decentralized Autonomous Organization (DAO) code issue lead to a $60 Million Ether theft during ICO (Initial Coin Offering). While part of this money was recovered by changing the ledger state with a special mechanism called a *Hard fork*, it is unlikely to be done for an issue faced by a few users.
- Most of these issues are due to immature SC code. Hence, we also discuss these topics in detail in the course.

Busting Blockchain Myths

Myth	Reality
Solves every problem	A kind of database with attached logic
Trustless	Can shift trust & spread trust
Secure	Focus is Integrity, not Confidentiality
Smart contracts are legal contracts	May help execute parts of legal contracts
Immutable	Many only offer probabilistic immutability/finality
Are inherently unscalable	Emerging blockchains are more scalable
Need to waste electricity	Emerging blockchains are more efficient
If beneficial, will be adopted	Adopting disruptive technology is tricky

- There are those who believe in BCs and those who don't. Most of this could be attributed to the involvement of cryptocurrencies and the negative publicity that they have gained.
- However, it is important that understand that BCs can function without cryptocurrencies, especially in enterprise applications.

- Here are some of the myths about BC:
  - BC can't solve every problem. It is more of a database and a computational engine to be used when there are concerns around trust.
  - A blockchain is not completely trustless. Instead, it shifts and spreads trust to a wider system consisting of a set of nodes and codes that govern the behaviour of those nodes. Trustless is a misnomer.
  - From the Confidentiality, Integrity, and Availability (CIA) triad of security, BCs focus is on integrity, not confidentiality.
  - SCs may help execute parts of some legal contracts but don't have the same regal acceptance.
  - As you will see later, immutability is probabilistic, which means it can change under very specific conditions.
  - While it's true that public BCs are unscalable and waste electricity, emerging designs are far more scalable and efficient. E.g., Ethereum 2.0. Some public BCs even claim they are carbon negative.
  - While BCs are certainly beneficial, adoption can be hampered by many non-technical factors.

3 Blockchain adoption Paradoxes

More Transparency! But Not About Me!

- Bitcoin relies on pseudonymous addresses; regulators concern about no KYC
- But public blockchains are too transparent for many commercial applications

Consensus Does Not Give You Agreement (to adopt blockchain)

- Who pays for a decentralised system? Who chooses policy? (including privacy)

Future of Blockchain Adoption is Off-Chain

- Business models, Decentralised governance, Law/Regulation, …
- "Parity problems" – Digital/physical, digital/legal
- Lots of technical innovation happening in "layer 2" & interoperability
- Value of blockchain is the difference it can make to our lives off-chain

- When we try to apply BC in real-world applications we have to deal with several paradoxes. These are perhaps the 3 key ones:

- Users want transparency about what others do, but not so much what they do.
- While BC addresses are pseudonymous, it has been shown that they can be linked given some additional details.
- Also, while BC users want anonymity, regulators want to know who's behind a transaction (KYC – Know Your Customer).
    - KYC involves at least collecting name, DOB, and address using documents such as driving licenses, passports, national identity cards, birth certificates, and utility bills, e.g., 100 points of ID based on primary and secondary documents in Australia.
- Public BCs are too transparent for many business applications. Private ones are better but still, there are some privacy concerns.

- On-chain consensus about blocks/TXs does not necessarily translate well into business agreements between parties.
- Also, there are other conflicts like who will host the BC, pay for it, or decide what should be the smart contract logic or who can update a contract.
- We will touch on this topic in the last class.

- As we saw, BC is likely to be part of a greater system. In that case, what happens off-chain would have a significant impact on what happens on-chain and broader application.
- Business models, governance, and laws and regulations are external.
- Also, how do you ensure what exists in the physical world link-up nicely with BC's virtual world? This is called the digital-physical parity problem.
- Many technical changes happen outside platforms that are connected to BC which are called layer 2 solutions. These provide fast transactions that can eventually be linked back to BC.
- Essentially, the true value of BC lies in the difference it can make in our lives outside BC. Other than in cryptocurrencies, BCs need some connection to off-chain components to be useful.

Question

Mark True or False for each the following statements about a blockchain's ability to handle data

	True	False
Blockchains can store a large volume of data		✓
Data can be read faster, but writing is slow	✓	
Data on a blockchain is secure		✓
It's impossible to correct errors in data		✓
Both data & rules that govern the manipulation of that data can be specified in a smart contract	✓	

Here's another quick question on data on BCs.

- Blockchains are not good for storing a large volume of data due to the high level of replication and global distribution of data.
- Data can be read faster, but writing is slow. Can read data from any node. But writes (i.e., any update to ledger state) must be through a TX that needs to get included in a block.
- Data on a blockchain is not considered to be secure as it lacks confidentiality
- It's possible to correct errors in data, e.g., Ethereum hard form due to DAO attack, but very difficult as your need coordination among many miners/validators. This is one of those myths
- Both data & rules that govern the manipulation of that data can be specified in a smart contract. This is essentially the idea of tokens and NFTs (non-fungible tokens).

Homework

1. Watch the Anders Brownworth video!
2. Explore the following terms

   a) Bitcoin Halving
   b) Ethereum (ETH) vs Ethereum Classic (ETC)
   c) Bitcoin (BTC) vs Bitcoin Cash (BCH)
   d) Cryptocurrency Wallet and Exchanges